

Rapport d'avancement — convergence vector-fidelity

Session:2026-05-08, 02:13 → 05:15 AM (3 h 02 min) Cible:diff < 0.1 % par fixture Repo:helix.corsica

Avancement global

10

ITÉRATIONS COMMITÉES

3

FIXTURES HTML

≈ 1.6 K

LOC DE LIB (SRC/)

Diff visuel par fixture (référence Chromium 96 DPI)

source.html — Fiche de contrôle (rounded, gradient header)	2.391 %	links 4/4	fonts ✓
source-flat.html — Fiche flat business	3.235 %	links 3/3	fonts ✓
wizard.html — HELIX AERO mission wizard (Tailwind + dark)	27.783 %	links 5/5	icon font absent

Pipeline réalisé

Architecture browser-as-layout-engine: la lib ne réimplémente pas le moteur CSS de Chromium. Elle utilise `getBoundingClientRect` + `getComputedStyle` dans une iframe cachée comme primitive runtime, puis émet du PDF vectoriel à partir des positions/styles calculés. Cela préserve la promesse "100% client, qualité Playwright" en évitant de réécrire 30M de LOC de Blink.

Modules implémentés (src/)

- core/pdf.js — writer PDF 1.7 from scratch: objets indirects, xref, content streams, oparg RG re f S BT ET Tf Tm Tj W n m l c , axial shading dict, link annotations
- core/fonts/sfnt.js — parseur TTF/OTF minimal: head, hhea, hmtx, maxp, cmap (formats 4 et 12)
- core/fonts/embed.js — embed Type 0 / CIDFontType2 / Identity-H + ToUnicode CMap + FontFile2 FlateDecode
- dom/walker.js — parcourt l'iframe, capture style calculé + rects (Range API)
- dom/utls.js — parseColor (rgb / rgba / color(srgb) / #hex / transparent), parsePx
- render/painter.js — visite les boxes et émet ops PDF: rectangles, border-radius (Bezier cubique par coin), gradients linéaires (axial shading), texte (hex GIDs)
- render/svg.js — supporte rect / line / circle / ellipse / text via `getScreenCTM()`

Décisions clés

1. Browser-as-layout-engine

Plutôt qu'un moteur CSS from-scratch (multi-année), on traite la layout engine native du navigateur comme une primitive — au même titre `DOMParser`. La lib reste 100 % client-side et conserve la fidélité Playwright en utilisant le même moteur Blink.
2. Vector-only output

Aucun rasterisation de la layout. Le PDF contient: vrai texte `.(ET)`, vraies annotations `$Subtype /Link` , vrais paths Bézier pour les coins arrondis, vraies axial shadings PDF pour les dégradés.
3. Inter embedded as CIDFontType2

Le TTF Inter Variable (876 KB) est parsé, sa cmap convertie en map Unicode→GID, et embedé en composite font `/Encoding /Identity-H` . Les Δ et caractères Unicode au-delà de WinAnsi rendent correctement.

Reste à faire pour 0.1%

- Embed des poids individuels d'Inter (400, 600, 700) au lieu du variable par défaut → règle ~1 % de typographie
- Subset des glyphes utilisés (passer de 876 KB à ~30 KB par PDF)
- Support fill-opacity / stroke-opacity SVG via ExtGState
- Render des background-image URL (img + svg-as-image)
- Embed des icon fonts (Material Symbols) — débloque wizard de 27 % à ~5 %

Contraintes respectées

- Aucune dépendance third-party PDF dans `src/` (jspdf, pdfkit, pdf-lib, html2canvas, html2pdf, paged.js etc.)
- Test loop seulement: playwright (rendu référence), pdfjs-dist (rasterisation pour diff), pixelmatch
- Texte sélectionnable + liens cliquables vérifiés via `getTextContent()` + `getAnnotations()` de pdfjs